

# kmhelpers: A Python toolkit for automated management of genomic indexes

Sébastien Bellenous<sup>1</sup> and Pierre Peterlongo <sup>1</sup>

<sup>1</sup> Genscale, Univ. Rennes, Inria, CNRS, IRISA - UMR 6074, Rennes, F-35000 France

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

## Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

## Summary

Large-scale genomic sequence search has become a central challenge in modern bioinformatics [REFSTODO]. A family of scalable methods uses Bloom filters (BF) (Bloom, 1970). At query time, this enables assigning the queried  $k$ -mers (words of length  $k$ ) to the set of datasets they belong to. Building such indexes on a vast set of genomic samples requires several steps. These steps require expertise, and any error at any stage can invalidate downstream results, waste significant computation time, and/or generate an oversized index.

We propose kmhelpers, an open-source Python toolkit that automates the entire workflow for building (and querying) indexes created using kmindex (Téo Lemané & others, 2024). The kmhelpers tool provides both a command-line interface (CLI) and a Python API, covering every stage of the  $k$ -mer index lifecycle.

## Statement of Need

$k$ -mer based sequence databases built with tools like kmindex (Téo Lemané & others, 2024) and kmtricks (Thibault Lemané et al., 2022) are used to address large-scale questions in genomics, metagenomics, and population studies. For instance, those tools were recently used to index 50 petabytes of raw sequencing data (Chikhi et al., 2025). However, the gap between having the raw indexing software and running a reproducible, end-to-end optimized indexing workflow is significant, and accessible only to specialists.

kmhelpers aims to popularize and make accessible the complete process of using kmindex for building or updating a genomic search engine from raw data. This novel possibility opens the door for non-specialists to the creation and maintenance of private indexes (such as in hospitals or data producer centers) or shared indexes that can be registered in larger search engines.

Formally, the input is a set  $\mathcal{S} = S_1, \dots, S_n$  of  $n$  genomic samples of various sizes. Each  $S_i$  can be either a raw sequencing dataset or assembled sequences. The output is an index built using kmindex, subject to two main user-defined limits: (1) the maximum allowed false-positive rate, and (2) the maximum number of sub-indexes, each being a file storing a set of BFs as a matrix. kmhelper orchestrates all steps (described below) and automatize and simplify the parametrization and the data distribution.

## Sub-indexes specific need

One needs to briefly explain the concept of sub-index, whose construction and parametrisation can be painful for non-specialists. Any Bloom filter-based index compacts all BFs of the same size as a (row-major) matrix in a single file. In such a matrix, a column is a BF and each row has length  $n$ , representing the presence (1) or absence (0) of a  $k$ -mer across all  $n$  samples, assuming the same hash function is used for all BFs. Finally, a complete index of a set of

39 samples is decomposed into distinct matrices — each being a sub-index — each storing all  
40 Bloom filters of the same size.

41 The main challenge arises from the fact that each BF size must be adapted to the number of  
42 items it contains. In the context of this work, a BF indexes all distinct  $k$ -mers from a sample  
43  $S_i$ . Suppose first that all samples possess the same number of distinct  $k$ -mers: all BFs would  
44 then have the same size and could be merged into a single matrix. This is the ideal situation,  
45 as only a single file needs to be accessed and opened at query time, and a row of this matrix  
46 directly provide the presence/absence of a queried  $k$ -mer in the  $n$  indexed samples.

47 In practice, sample sizes differ from one another. One solution would be to adapt the BF size  
48 to the largest sample. However, sample sizes can vary by several orders of magnitude, which  
49 would result in enormous storage overhead. The opposite extreme would involve creating  
50 one matrix per sample (each composed of a single column). This would be optimal in terms  
51 of storage size, but would dramatically slow down the query process, as  $n$  (can be at the  
52 scale of several millions) distinct files would need to be accessed for each queried  $k$ -mer. The  
53 solution we propose is a middle ground. The user defines the maximum number of sub-groups  
54 — hence the maximum number of matrices created — and, as shown in the pipeline below,  
55 kmhelpers automatically determines the BF size for each sub-index such that the total index  
56 size is minimised.

## 57 Whole pipeline needs

58 Let us now take a step back and look at the entire pipeline, automatically processed by the  
59 kmhelper pipeline. The identified needs follow the following steps:

- 60 ▪ Sample discovery and  $k$ -mer counting (`list`). This step recursively lists all samples from  
61 a given directory and counts the  $k$ -mers of each datasets using ntCard (Mohamadi et  
62 al., 2017) (unless these number are provided by the user).
- 63 ▪ Bloom-filter sub-indexes sizes profiling (`profile`). This step determines the best set of  
64 sizes of sub-indexes to built, given a maximal number of them, provided by the user.
- 65 ▪ Index definition generation (`compose`). This step enables to distribute each input sample  
66 to its correct sub-index, and to prepare the *files of files* describing the origine of data of  
67 each sub-indexes.
- 68 ▪ Ppfront validation of sample files, available disk space and memory with generation of  
69 ready-to-execute pipeline scripts (`plan`).
- 70 ▪ Index building from definition files (`apply`). This step builds all sub-indexes.
- 71 ▪ ZSTD-based index (block) compression (`compress`) (?). This an optional step that  
72 applies a specific compression skeme... todo
- 73 ▪ registry management (`registry`). todo

74 In addition, kmhelper can perform queries, once the index is built.

75 Multi-step workflows can be described as declarative YAML pipelines (`pipeline`) and executed  
76 in a single command.

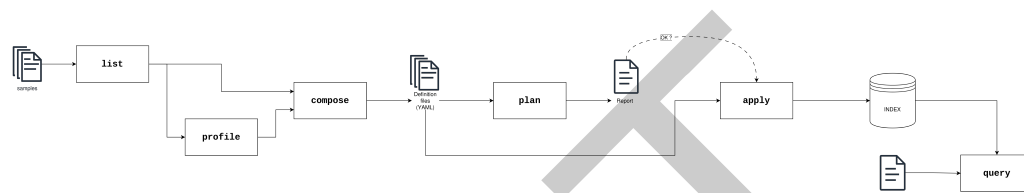
## 77 State of the Field

78 Several tools address large-scale sequence search using  $k$ -mer based data structures. BIGSI  
79 (Bradley et al., 2019) and COBS (Bingmann et al., 2019) use compressed Bloom filter matrices  
80 to answer presence/absence queries across collections of sequencing experiments. HowDeSBT  
81 (Harris & Medvedev, 2020) and Mantis (Pandey et al., 2018) use sequence Bloom trees and  
82 counting quotient filters, respectively, to support similar queries. kmindex, which kmhelpers  
83 wraps, builds on the kmtricks counting pipeline and is designed for efficient querying of large  
84 sequence collections.

85 kmhelpers does not compete with any of these approaches at the algorithmic level. Its

contribution is orthogonal: it provides the workflow automation, parameter optimisation, and index lifecycle management that are necessary to deploy kmindex-based databases in practice, but are absent from the core tool. Similar automation layers exist for other complex bioinformatics pipelines (e.g., Snakemake (Köster & Rahmann, 2012) workflows wrapping alignment or assembly tools), but no dedicated automation layer existed for kmindex prior to kmhelpers.

## Design and Implementation



**Figure 1:** Overview of the kmhelpers workflow. `list` enumerates sample files and counts  $k$ -mers; `profile` analyses the count distributions to assign each sample to a Bloom-filter span and recommend index groupings. Both outputs feed into `compose`, which generates YAML index definition files. `plan` then validates sample paths, available disk space and memory, and emits a ready-to-execute pipeline script; the resulting report is reviewed before committing to the build. `apply` reads the definition files and invokes kmindex to construct the index. Finally, `query` searches the built index against user-provided sequences and returns ranked results.

`#todo: ajouter la mise à jour d'un index, à la fois dans la figure et dans l'outil.`

kmhelpers is implemented in Python ( $\geq 3.8$ ) and distributed via Conda with automatic installation of its bioinformatics dependencies (kmindex, ntCard). The CLI is built with Click (Ronacher & Click Contributors, 2023) and the package exposes a public Python API covering all CLI functionality.

**Workflow orchestration.** Building a large index may involve hundreds to millions of samples spread across multiple sub-indexes. Managing these build jobs manually is error-prone. kmhelpers introduces a declarative index definition format (YAML) and two complementary commands: `plan`, which validates sample files, available disk space and memory upfront and emits ready-to-execute pipeline scripts; and `apply`, which reads the definition files and runs the build, with support for span-level and name-level filtering. [`#Commentaire Pierre : quelles sont les nouvelles infos que tu souhaites faire passer ici. Ne faut il pas une bonne fois pour toute développer les étapes (dans Whole pipeline needs), et ne pas revenir dessus ?`]

Together, these features make  $k$ -mer indexing workflows accessible to researchers who are not experts in the underlying data structures, while remaining flexible enough for large-scale production use.

**ToL ?**

`#TODO`

## Availability and documentation

`#TODO`

## 113 Acknowledgements

114 The authors thank Téo Lemane for developing kmindex and for his responsiveness in  
 115 addressing feature requests and issues raised during the development of kmhelpers. We  
 116 acknowledge the GenOuest core facility (<https://www.genouest.org>) for providing the  
 117 computing infrastructure. The work was funded by the Inria Challenge “OmicFinder”  
 118 (<https://project.inria.fr/omicfinder/>).

## 119 AI Usage Disclosure

120 AI assistance was used for constrained tasks (drafting, editing, code suggestions) under strict  
 121 human review at every stage. AI provider used: Claude (Anthropic, 2025).

## 122 Removed text (to remove before submission)

123 \$- #  $k$ -mer indexes built on Bloom filters allow efficient querying of large sequence collections.  
 124 However, building and managing such indexes in practice involve a complex chain of steps.

125 \$- # such as those produced by kmindex (Téo Lemane & others, 2024), : enumerating  
 126 sample files, counting  $k$ -mers, selecting appropriate Bloom-filter parameters, generating index  
 127 definitions, orchestrating build jobs, and compressing outputs for long-term storage. Each  
 128 step requires expertise, and errors at any stage can invalidate downstream results and waste  
 129 significant computation time.

130 **Parameter selection.** Bloom filter-based indexes require choosing a filter size (or *span*) per  
 131 sample. Choosing this parameter incorrectly leads to either excessive storage consumption or  
 132 unacceptably high false-positive rates. kmhelpers automates this with its profile command,  
 133 which reads  $k$ -mer count distributions produced by ntCard (Mohamadi et al., 2017) and  
 134 assigns each sample to the smallest span that keeps the false-positive rate below a user-defined  
 135 threshold.

136 The complete pipeline can be described as follows. First, determine the length of each sample  
 137 (in terms of its number of distinct  $k$ -mers). Second, given a user-defined false positive rate,  
 138 distribute input samples into distinct groups, each BF of a group having the same fixed size.  
 139 The sizes of BF of groups have to be optimized to reduce the final whole index size. Third,  
 140 given computational constraints, the whole construction is split into jobs, each generating  
 141 sub-parts of the index, and a final step merges these sub-parts.

## 142 References

- 143 Bingmann, T., Bradley, P., Gauger, F., & Iqbal, Z. (2019). COBS: A compact bit-sliced  
 144 signature index. *String Processing and Information Retrieval (SPIRE 2019)*. [https://doi.org/10.1007/978-3-030-32686-9\\_21](https://doi.org/10.1007/978-3-030-32686-9_21)
- 146 Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors.  
 147 *Communications of the ACM*, 13(7), 422–426.
- 148 Bradley, P., Den Bakker, H. C., Rocha, E. P. C., McVean, G., & Iqbal, Z. (2019). Ultrafast  
 149 search of all deposited bacterial and viral genomic data. *Nature Biotechnology*, 37(2),  
 150 152–159. <https://doi.org/10.1038/s41587-018-0010-1>
- 151 Chikhi, R., Lemane, T., Loll-Krippelbein, R., Montoliu-Nerin, M., Raffestin, B., Camargo, A. P.,  
 152 Miller, C. J., Fiamenghi, M. B., Agostinho, D. P., Majidian, S., & others. (2025). Logan:  
 153 Planetary-scale genome assembly surveys life's diversity. *bioRxiv*, 2024–2007.

- 154 Harris, R. S., & Medvedev, P. (2020). Improved representation of sequence Bloom trees.  
155 *Bioinformatics*, 36(3), 721–727. <https://doi.org/10.1093/bioinformatics/btz662>
- 156 Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine.  
157 *Bioinformatics*, 28(19), 2520–2522. <https://doi.org/10.1093/bioinformatics/bts480>
- 158 Lemane, Thibault, Medvedev, P., Chikhi, R., & Peterlongo, P. (2022). Kmtricks: Efficient and  
159 flexible construction of Bloom filters for large sequencing data collections. *GigaScience*, 11,  
160 giac029. <https://doi.org/10.1093/gigascience/giac029>
- 161 Lemane, Téo, & others. (2024). Indexing and real-time user-friendly queries in terabyte-sized  
162 complex genomic datasets with kmindex and ORA. *Nature Computational Science*, 4(2),  
163 104–109. <https://doi.org/10.1101/2023.05.31.543043>
- 164 Mohamadi, H., Khan, H., & Birol, I. (2017). ntCard: A streaming algorithm for cardinality  
165 estimation in genomics data. *Bioinformatics*, 33(9), 1324–1325. <https://doi.org/10.1093/bioinformatics/btw832>
- 166
- 167 Pandey, P., Almodaresi, F., Bender, M. A., Ferdman, M., Johnson, R., & Patro, R. (2018).  
168 Mantis: A fast, small, and exact large-scale sequence-search index. *Cell Systems*, 7(2),  
169 201–207.e4. <https://doi.org/10.1016/j.cels.2018.05.021>
- 170 Ronacher, A., & Click Contributors. (2023). *Click: Python composable command line interface*  
171 *toolkit*. <https://click.palletsprojects.com>